

AuraSense — Architecture & Feature Plan

Document purpose: detailed system architecture and a practical feature plan/actionable task list to prepare a strong hackathon prototype for the Pfizer & Aava challenge (Predictive Migraine Detection).

1. High-level architecture (Mermaid)

```
flowchart TD
    subgraph Mobile
        MApp["Mobile App\n(React Native)"]
        MHK["HealthKit / Health Connect"]
        MCal["Calendar API"]
    end

    subgraph Backend
        API["FastAPI"]
        Auth["Auth (Firebase Auth / OAuth)"]
        Ingest["Ingestion Queue (Redis/Celery)"]
        ModelSrv["Model Service (Torch/XGBoost)"]
        Features["Feature Store (Firestore / PostgreSQL)"]
        Explain["Explainability (SHAP)"]
        Notif["Notification Service (APNS / FCM)"]
    end

    subgraph External
        Weather["(OpenWeatherMap / Tomorrow.io)"]
    end

    MApp -->|Health data, calendar, user inputs| API
    MApp -->|Feedback & logs| API
    API --> Auth
    API --> Ingest
    Ingest --> ModelSrv
    ModelSrv --> Features
    ModelSrv --> Explain
    ModelSrv --> API
    API --> Notif
    API -->|weather request| Weather
    Weather --> API
```

2. Component responsibilities

- **Mobile App (React Native)**
- Collect health metrics (via HealthKit / Health Connect) or use mocked data.

- Collect user input (coffee, skipped meal, stress) and feedback logging.
- Request calendar permissions and send summarized schedule features.
- Show risk score, contributing factors (cards), and prevention suggestions.

- Send push notifications for high risk.

• **FastAPI Backend**

- Auth + user management.
- Ingest batched features from mobile (or simulated clients).
- Orchestrate feature extraction and call model inference.
- Store user history and feedback for later fine-tuning.
- Serve SHAP explanations / human-readable contributing factors.

• **Model Service**

- Lightweight inference endpoint (either inside FastAPI or as separate microservice).
- Trained offline on synthetic/real datasets.
- Returns probability for next 12 hours and feature importances.

• **Feature Store**

- Time-series storage for user metrics (short retention for demo is fine).
- Example: Firestore collections or PostgreSQL with timestamped rows.

• **Queue (Celery + Redis)**

- For asynchronous heavy tasks: feature engineering, batch predictions, model retraining.

• **Explainability**

- SHAP or permutation importance to translate model output into "top contributing factors".

• **External APIs**

- Weather API for pressure, humidity, temperature.
- Calendar API (mobile) — aggregate local event density / stress proxy.

3. Data model (example)

User: { user_id, name, timezone, created_at }

Metrics (time-series): - user_id, timestamp, sleep_hours, sleep_quality, steps, avg_hr, hrv, activity_minutes

Context features: - `pressure`, `temperature`, `humidity`, `weather_code`, `calendar_event_count`, `calendar_stress_score`

UserInputs: - `caffeine_bool`, `meal_skipped_bool`, `self_reported_stress` (0-3), `notes`

Label: - `migraine_event` (bool), `onset_timestamp`, `pain_severity` (1-10)

4. API endpoints (MVP)

- `POST /api/v1/ingest` — Accepts batched features from mobile `{user_id, metrics[], context, inputs}`
 - `GET /api/v1/predict?user_id=...` — Returns `{probability, top_factors[], recommendations[]}`
 - `POST /api/v1/feedback` — `{user_id, migraine_event, severity, notes}`
 - `POST /api/v1/register` — create user (via OAuth / Firebase Auth)
 - `GET /api/v1/history?user_id=&from=&to=` — returns recent time-series + events
-

5. Model design (initial baseline)

Objective: probabilistic prediction for migraine occurrence within next 12 hours.

Baseline model (fast to prototype): - Gradient boosting (XGBoost / CatBoost) on engineered tabular features (windowed aggregates: `sleep_last_3d_mean`, `pressure_delta_6h`, `steps_last_24h`, `caffeine_today`)
- Train/test split with user-wise CV (time-aware) - Metrics: ROC-AUC, Precision@k, Recall (emphasise recall), Calibration (Brier score)

Stronger variant if time allows: - Temporal model (Temporal Fusion Transformer or small Transformer/LSTM on sequences per user) - Personalization: fine-tune small head per-user or include user embeddings

Explainability: SHAP for per-prediction explanations (top 3 contributors)

6. Feature plan & timeline (Hackathon-oriented)

Pre-hackathon (1–2 weeks before)

- [] **Repo scaffold & templates** (monorepo with `mobile/`, `backend/`, `model/`) — assign owners
- [] **Synthetic dataset generator** (Python script to simulate users with triggers)
- [] **Baseline model** (XGBoost) trained on synthetic data — save model artifacts
- [] **FastAPI skeleton** with `/predict` endpoint that loads model
- [] **React Native UI mockups** (Figma or in-app static screens)
- [] **Demo script & slides skeleton**

Hackathon Day 1 — Core infra & ingestion

- [] Mobile: basic app with forms + mock data sender to backend
- [] Backend: `/ingest`, `/predict` endpoints working locally
- [] Weather API integration; calendar mock extractor
- [] Model inference hooked up; basic frontend shows risk

Hackathon Day 2 — UX, explainability, feedback

- [] SHAP integration to show top_contributors
- [] Recommendations engine (rule-based) + human readable text
- [] Feedback endpoint; store label events
- [] Push notifications for high-risk events
- [] Polish UI and onboarding flow

Hackathon Day 3 — Polish & pitch

- [] Demo user flows (2–3 scenarios) scripted and rehearsed
 - [] Finalize dashboard & visuals (gauge, timeline, factors)
 - [] Add minor personalization (simple calibration by feedback)
 - [] Prepare 3–5 minute pitch: problem, demo, results, future roadmap
-

7. Team roles & recommended split

- **Product / PM (1):** Story, demo script, pitch deck
 - **Mobile dev (1–2):** React Native app, Health API mocks/integration
 - **Backend dev (1):** FastAPI, Auth, endpoints, deployment
 - **ML engineer (1–2):** synthetic data generator, baseline model, SHAP explanations
 - **Designer / Frontend (optional):** polish UI, graphics for pitch
-

8. Demo scenarios (3 quick flows)

1. **New user morning:** app fetches sleep (5h), weather shows 10 hPa pressure drop → app warns: High risk next 12h; top factors: low sleep, pressure drop. Recommendation: rest, hydrate.
 2. **Workday overload:** calendar has 6 back-to-back meetings; heart rate elevated → app warns: Moderate risk; suggests short walk + reduce caffeine.
 3. **Feedback loop:** user reports migraine despite recommendation → model logs event and app shows that it will adapt over time.
-

9. Privacy & ethical notes (quick)

- Minimum data collection: only features required for prediction. Support local-only mode (all inference on-device) as future work.
 - Use hashed user IDs and encryption for stored data.
 - Be explicit in pitch about *simulated data* during demo and steps required for real-world validation and IRB/ethical approvals for clinical use.
-

10. Success metrics for the hackathon

- Working end-to-end demo: mobile -> backend -> predict -> explain (required)
 - Interpretable explanations: top 3 contributors shown for each prediction
 - UX clarity: user understands next steps to reduce risk
 - Technical polish: stable demo, minimal latency (<2s for prediction)
 - Story & impact: clear demonstration of how this helps a real user
-

11. Extras & stretch ideas (if time)

- On-device inference via TensorFlow Lite / CoreML small model
 - LLM-generated personalized micro-interventions (integrate Llama/LLM prompt for friendly recommendations)
 - Multi-user demo with leaderboard showing reduction in migraine frequency (simulated)
 - Map pressure patterns geographically (privacy-conscious aggregate)
-

Appendix: Quick commands / references

- XGBoost / CatBoost basics: `pip install xgboost catboost`
- SHAP quickstart: <https://github.com/slundberg/shap>
- OpenWeatherMap API: <https://api.openweathermap.org/data/2.5/weather>
- HealthKit doc: <https://developer.apple.com/documentation/healthkit>
- Health Connect: <https://developer.android.com/guide/health-and-fitness/health-connect>

End of document.